Rexell AI Bot Detection & Blockchain Integration Plan

1. Executive Summary

This document presents a comprehensive engineering plan for upgrading the Rexell event ticketing platform's AI bot detection module from a rudimentary client-side heuristic system to a production-grade, multi-layered, server-side ML pipeline integrated directly with the Celo blockchain. The current system relies on hardcoded logistic regression weights executed in the browser with purchase history stored in `localStorage`—an architecture trivially bypassed by sophisticated bots. The proposed architecture introduces a four-layer detection pipeline (real-time heuristic filter → behavioral CNN → session-level graph neural network → ensemble decision engine), server-side API infrastructure, behavioral biometrics collection, device fingerprinting, on-chain wallet analysis, and a novel `AIOracle` smart contract that enforces AI-driven risk decisions on-chain before ticket purchases execute. This plan draws from state-of-the-art research including the PureNFT lightweight CNN architecture, multi-stage bot detection pipelines using SGANs and DGCNNs, decentralized federated learning for EVM bot detection, and behavioral sequence analysis achieving 98.52% accuracy. The implementation spans 14 weeks across five phases, targeting a detection accuracy above 95% with sub-200ms inference latency.

2. Current State Analysis

2.1 Architecture Overview

Rexell's current AI module consists of five components executing entirely in the browser. The `BotDetector` uses a hardcoded sigmoid function $\sigma(-1.5 \cdot t_{norm} + 3.0 \cdot f_{norm} - 0.5)$ with fixed weights—not a trained model. The `ScalpingDetector` applies simple threshold rules on per-event purchase counts. The `RiskEvaluationAgent` combines these two scores linearly, the `PolicyEnforcementAgent` maps the result to ALLOW/WARNING/BLOCK, and `AIModeService` orchestrates the pipeline using `localStorage` for state.

2.2 Critical Weaknesses

Weakness	Impact	Severity
Client-side execution	Bots can intercept and modify detection logic via browser DevTools	Critical
`localStorage` state	Purchase history is trivially manipulated or cleared	Critical
Hardcoded weights (not ML)	No generalization; fixed decision boundary cannot adapt to new attack patterns	High
Only 2 bot features (time delta, frequency)	Misses behavioral, device, network, and on-chain signals	High

Weakness	Impact	Severity
Only 1 scalping feature (event repetition)	Cannot detect distributed scalping across wallets	High
No behavioral biometrics	Cannot distinguish human interaction patterns from bot automation	High
No device fingerprinting	Cannot identify device spoofing or headless browsers	Medium
No IP/network analysis	Cannot detect proxy farms or VPN-masked traffic	Medium
No on-chain data integration	Ignores wallet history, transaction patterns, and cross-event behavior	High
No smart contract enforcement	AI decisions are advisory only; contract executes regardless	Critical
No CNN model deployed	The Python TensorFlow/CNN code exists but is not integrated into the buy flow	High

Research on web bot evasion using GANs demonstrates that bots can generate human-like mouse movements to bypass single-layer behavioral checks, underscoring the need for multi-layered defense. Ticket scalping analysis confirms that modern scalper bots complete purchases in under 100ms, rendering the current time-delta-only approach insufficient.

3. Proposed Architecture

3.1 High-Level System Design

The new architecture shifts all detection logic server-side and introduces a four-layer ML pipeline with blockchain enforcement:

```
┌──────────────────────────────────────────────────────────────┐
│                    FRONTEND (Next.js)                        │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────┐   │
│  │ Behavioral   │  │ Device       │  │ Purchase Flow    │   │
│  │ Collector    │  │ Fingerprinter │  │ (Modified)       │   │
│  └──────┬───────┘  └──────┬───────┘  └────────┬─────────┘   │
└─────────┼─────────────────┼──────────────────┼──────────────┘
          │                 │                  │
          ▼                 ▼                  ▼
┌──────────────────────────────────────────────────────────────┐
│                SERVER-SIDE AI API (Next.js API Routes)       │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────────┐  │
│  │ Layer 1: │  │ Layer 2: │  │ Layer 3: │  │ Layer 4:     │  │
│  │ Heuristic │→│ Behavioral│→│ Graph    │→│ Ensemble     │  │
│  │ Filter   │  │ CNN      │  │ NN       │  │ Engine       │  │
│  └──────────┘  └──────────┘  └──────────┘  └──────┬───────┘  │
│                                                    │         │
│  ┌──────────────────────────┐  ┌──────────────────┴──────┐  │
│  │ Feature Store (Redis/DB) │  │ AI Decision API         │  │
│  └──────────────────────────┘  └──────────────────────────┘  │
└──────────────────────────────────────────────────────────────┘
                                                   │
                                                   ▼
```

```
┌──────────────────────────────────────────────────────────┐
│                    CELO BLOCKCHAIN                       │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────┐  │
│  │ AIOracle.sol │  │ Rexell.sol   │  │ SoulboundID.sol  │  │
│  │ (risk scores │  │ (buyTicket   │  │ (KYC score       │  │
│  │  + approvals)│  │  w/ AI check) │  │  updates)        │  │
│  └──────────────┘  └──────────────┘  └──────────────────┘  │
└──────────────────────────────────────────────────────────┘
```

3.2 Data Flow for a Ticket Purchase

1. **Collection Phase**: As the user browses the event page, the `BehavioralCollector` React component silently captures mouse movements, keystroke dynamics, scroll patterns, and touch events. The `DeviceFingerprinter` generates a stable device hash using canvas, WebGL, AudioContext, and navigator properties.
2. **Pre-Purchase Assessment**: When the user clicks "Buy Ticket," the frontend sends a `POST /api/ai/assess-risk` request containing the behavioral telemetry, device fingerprint, wallet address, and event ID.
3. **Server-Side Pipeline**: The API route runs the four-layer pipeline sequentially (with early termination if Layer 1 flags obvious bots). On-chain data for the wallet is fetched via Celo RPC.
4. **Decision & Signing**: The ensemble engine produces a risk score (0–1) and action (ALLOW/WARN/BLOCK). For ALLOW, the server generates a signed approval message (EIP-712) containing the risk score.
5. **On-Chain Enforcement**: The frontend submits the signed approval alongside the `buyTicket()` transaction. The `AIOracle` modifier in `Rexell.sol` verifies the signature and checks that the risk score is below the threshold before allowing execution.
6. **Post-Purchase Update**: After a successful purchase, the `SoulboundIdentity` KYC score is optionally adjusted based on accumulated behavioral trust.

4. Feature Engineering Pipeline

4.1 Behavioral Biometrics

Behavioral biometrics provide continuous authentication signals that are difficult for bots to replicate convincingly. The following features are collected via a lightweight React hook:

Feature Category	Specific Features	Collection Method
Mouse dynamics	Movement speed (px/ms), acceleration, curvature, jitter, straight-line ratio, pause frequency	`mousemove` events sampled at 60Hz
Keystroke dynamics	Key hold time, inter-key interval, typing speed (WPM), error rate, digraph latency	`keydown`/`keyup` events
Scroll behavior	Scroll velocity, acceleration, direction changes, smooth vs. discrete scrolling	`scroll` + `wheel` events

Feature Category	Specific Features	Collection Method
Touch patterns (mobile)	Touch pressure, contact area, swipe velocity, multi-touch frequency	`touchstart/touchmove/touchend`
Click patterns	Click precision (distance from element center), double-click speed, right-click frequency	`click/contextmenu` events
Gyroscope/Accelerometer	Device orientation changes, micro-movements during interaction	DeviceMotion API

BotShape's research demonstrates that converting these raw events into behavioral time series and analyzing their shape characteristics yields 98.52% detection accuracy.

4.2 Device Fingerprinting

Signal	Detection Purpose	Method
Canvas fingerprint	Headless browser detection	Render test shapes, hash pixel data
WebGL renderer/vendor	GPU spoofing detection	`WEBGL_debug_renderer_info` extension
AudioContext fingerprint	Automation framework detection	Oscillator node output hash
Navigator properties	Property inconsistency detection	`navigator.webdriver`, `plugins.length`, `languages`
Screen resolution + color depth	VM/emulator detection	`screen.width`, `screen.height`, `screen.colorDepth`
Timezone vs. IP geolocation	Proxy detection	Compare `Intl.DateTimeFormat` with IP-derived timezone
Font enumeration	OS fingerprinting	Measure rendered width of test strings

4.3 Network Analysis

- **IP reputation**: Query IP reputation databases (AbuseIPDB, IPQualityScore) for known bot/proxy IPs.
- **VPN/proxy detection**: Identify datacenter IPs, Tor exit nodes, and residential proxy services.
- **Geolocation consistency**: Flag mismatches between IP geolocation, browser timezone, and wallet transaction history locations.
- **Request rate limiting**: Track requests per IP with sliding window counters.

4.4 On-Chain Features

On-chain wallet analysis provides tamper-proof behavioral history that cannot be fabricated. BotDetect's federated learning approach validates the effectiveness of on-chain features for EVM bot detection.

Feature	Description	Predictive Signal
Wallet age	Block number of first transaction	New wallets correlate with bot farms
Transaction count	Total tx on Celo	Low-activity wallets are suspicious for high-value purchases
Token diversity	Number of distinct ERC-20 tokens held	Bots typically hold only cUSD
NFT holdings	Count of ERC-721 tokens across contracts	Indicates genuine collector behavior
DeFi interactions	Interactions with known DeFi protocols (Ubeswap, Moola)	Human users have diverse DeFi history
Funding source	Whether wallet was funded by a mixer, CEX, or known bot wallet	Mixer-funded wallets are high-risk
Cross-event behavior	Number of distinct events purchased across Rexell	Scalpers target high-demand events exclusively
Resale history	Ratio of tickets resold vs. attended	High resale ratio indicates scalping
Gas price patterns	Whether user consistently uses exact minimum gas	Bot-like gas optimization
Transaction timing	Entropy of transaction timestamps	Bots transact at regular intervals

4.5 Temporal & Event-Specific Features

Research on anomalous ticket purchasing identifies these critical temporal features:

- **Purchase velocity**: Time from event page load to `buyTicket()` execution (bots < 2 seconds).
- **Inter-purchase interval**: Time between consecutive purchases across events.
- **Session duration**: Total time on site before first purchase.
- **Price-to-time ratio**: Face value divided by time-to-sellout—a key predictor identified by PureNFT for scalper detection.
- **Demand ratio**: Tickets sold / total supply at time of purchase.
- **Seat selection pattern**: Sequential seat IDs indicate automated selection.
- **Payment method consistency**: Whether the same wallet uses multiple funding paths.

5. ML Model Architecture

5.1 Layer 1: Real-Time Heuristic Filter

The first layer applies deterministic rules with zero ML inference cost, catching the most obvious bots before they consume GPU resources. This approach follows the multi-stage pipeline architecture where heuristic pre-filtering reduces the load on expensive ML models.

```
// lib/ai/layers/heuristic-filter.ts
interface HeuristicResult {
  isBot: boolean;
```

```
  confidence: number;
  reason: string | null;
}

export function heuristicFilter(features: RequestFeatures): HeuristicResult {
  // Rule 1: WebDriver flag detected
  if (features.device.navigatorWebdriver === true) {
    return { isBot: true, confidence: 0.99, reason: 'WEBDRIVER_DETECTED' };
  }
  // Rule 2: Purchase velocity under 500ms (humanly impossible)
  if (features.temporal.pageLoadToPurchaseMs < 500) {
    return { isBot: true, confidence: 0.95, reason: 'SUPERHUMAN_SPEED' };
  }
  // Rule 3: No mouse movement events recorded
  if (features.behavioral.mouseEventCount === 0 && !features.device.isMobile) {
    return { isBot: true, confidence: 0.90, reason: 'NO_MOUSE_ACTIVITY' };
  }
  // Rule 4: Known datacenter IP
  if (features.network.isDatacenterIP) {
    return { isBot: true, confidence: 0.85, reason: 'DATACENTER_IP' };
  }
  // Rule 5: Canvas fingerprint matches known headless browser
  if (HEADLESS_CANVAS_HASHES.has(features.device.canvasHash)) {
    return { isBot: true, confidence: 0.92, reason: 'HEADLESS_CANVAS' };
  }
  // Rule 6: Wallet age < 1 hour and buying high-demand event
  if (features.onChain.walletAgeHours < 1 && features.event.demandRatio > 0.8) {
    return { isBot: true, confidence: 0.80, reason: 'FRESH_WALLET_HIGH_DEMAND' };
  }
  return { isBot: false, confidence: 0.0, reason: null };
}
```

Specification: Input = structured feature object, Output = binary + confidence, Latency target = < 5ms.

5.2 Layer 2: Behavioral CNN Model

Inspired by the PureNFT architecture that achieves 93.57% accuracy with only 22.82 KB model size and 0.0107 MFLOPs, the behavioral CNN converts collected features into a 2D feature map and applies factorized composite convolution (FCC) blocks.

Architecture:

```
Input: Feature tensor [batch, 1, 48, 48]
  ├── FCC Block 1: Conv1×3 → Conv3×1 → BatchNorm → ReLU  (16 filters)
  ├── FCC Block 2: Conv1×3 → Conv3×1 → BatchNorm → ReLU  (32 filters)
  ├── MSCGC Block: 4 parallel grouped convolutions → concat  (64 filters)
  ├── Global Average Pooling
  ├── Dense(128) → ReLU → Dropout(0.3)
  └── Dense(3) → Softmax  (human / bot / scalper)
```

The FCC blocks use cascaded 1×3 and 3×1 kernels instead of standard 3×3 convolutions, reducing parameters by ~33% while maintaining receptive field. The MSCGC block processes

feature subsets in parallel streams, capturing different aspects of behavioral patterns simultaneously.

Feature Map Construction: The 48×48 input matrix is constructed by arranging normalized features into a structured 2D grid:

- Rows 0–11: Behavioral biometrics (mouse speed histogram, keystroke intervals)
- Rows 12–23: Device + network features (one-hot encoded)
- Rows 24–35: Temporal features (time series of purchase intervals)
- Rows 36–47: On-chain features (wallet metrics, transaction patterns)

Specification: Model size ≈ 25 KB, Inference latency < 50ms on CPU, Input = [1, 48, 48] float32, Output = [3] softmax probabilities.

5.3 Layer 3: Session-Level Graph Neural Network

Following the DGCNN approach for session-wide behavior analysis, this layer constructs a wallet interaction graph to detect coordinated bot attacks (e.g., Sybil networks sharing funds from a common source).

Graph Construction:

- **Nodes**: Wallet addresses that have interacted with Rexell events.
- **Edges**: Weighted connections based on: (a) direct token transfers between wallets, (b) shared funding source, (c) similar purchase timing patterns (< 5s apart on same event), (d) identical device fingerprints.
- **Node features**: Per-wallet aggregated risk scores from Layer 2, on-chain metrics, purchase history.

Model: A 3-layer Graph Convolutional Network (GCN) with edge attention:

```
Input: Graph G = (V, E) with node features X ∈ R^{|V|×64}
  ├── GCN Layer 1: X' = ReLU(D^{-1/2} A D^{-1/2} X W_1)  → 64-dim
  ├── GCN Layer 2: X'' = ReLU(D^{-1/2} A D^{-1/2} X' W_2) → 32-dim
  ├── Graph Attention: α_ij = softmax(LeakyReLU(a^T [Wh_i || Wh_j]))
  ├── Readout: Global mean + max pooling over target node neighborhood
  └── MLP: Dense(32) → Dense(1) → Sigmoid  (coordination probability)
```

Specification: Latency < 100ms for subgraph of 500 nodes, Output = coordination probability [0, 1].

5.4 Layer 4: Ensemble Decision Engine

The ensemble engine combines outputs from all layers with the existing `RiskEvaluationAgent` and `PolicyEnforcementAgent` logic:

```
// lib/ai/layers/ensemble-engine.ts
interface LayerOutputs {
```

```
  heuristic: { isBot: boolean; confidence: number };
  cnn: { human: number; bot: number; scalper: number };
  gnn: { coordinationProb: number };
}

export function computeFinalRisk(outputs: LayerOutputs): RiskDecision {
  // Hard override: heuristic certainty
  if (outputs.heuristic.isBot && outputs.heuristic.confidence > 0.95) {
    return { action: 'BLOCK', riskScore: 0.99, reason: outputs.heuristic.reason };
  }

  // Weighted ensemble
  const botScore = outputs.cnn.bot * 0.45
    + outputs.heuristic.confidence * 0.25
    + outputs.gnn.coordinationProb * 0.30;

  const scalpScore = outputs.cnn.scalper * 0.50
    + outputs.gnn.coordinationProb * 0.30
    + outputs.heuristic.confidence * 0.20;

  const riskScore = Math.max(botScore, scalpScore);

  // Trust score (inverse of risk)
  const trustScore = 1 - riskScore;

  if (trustScore < 0.3) return { action: 'BLOCK', riskScore, reason: 'HIGH_RISK' };
  if (trustScore < 0.7) return { action: 'WARNING', riskScore, reason: 'MODERATE_RISK' }
  return { action: 'ALLOW', riskScore, reason: null };
}
```

Total Pipeline Latency Budget:

Layer	Target Latency	Notes
Layer 1: Heuristic	< 5ms	Pure rule evaluation
Layer 2: CNN	< 50ms	ONNX Runtime on CPU
Layer 3: GNN	< 100ms	Pre-computed graph, local subgraph inference
Layer 4: Ensemble	< 5ms	Weighted arithmetic
Signature generation	< 10ms	EIP-712 signing
Total	< 170ms	Within 200ms budget

6. Blockchain-AI Integration

6.1 AIOracle Smart Contract

The `AIOracle` contract bridges off-chain AI computation with on-chain enforcement, following the research gap identified in the PureNFT paper—embedding AI decisions architecturally into smart contracts rather than running them as separate components.

```
// contracts/AIOracle.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.17;

import "@openzeppelin/contracts/access/Ownable.sol";
import "@openzeppelin/contracts/utils/cryptography/ECDSA.sol";
import "@openzeppelin/contracts/utils/cryptography/EIP712.sol";

contract AIOracle is Ownable, EIP712 {
    using ECDSA for bytes32;

    address public aiSigner;  // Server's signing key
    uint256 public riskThreshold = 700;  // 0.7 * 1000, scaled to avoid floats
    uint256 public approvalValiditySeconds = 300;  // 5 minutes

    // Nonce per wallet to prevent replay attacks
    mapping(address => uint256) public nonces;

    // Risk score history for analytics
    mapping(address => uint256) public lastRiskScore;

    // Blacklisted wallets
    mapping(address => bool) public blacklisted;

    bytes32 private constant APPROVAL_TYPEHASH = keccak256(
        "AIApproval(address buyer,uint256 eventId,uint256 riskScore,uint256 nonce,uint256
    );

    event RiskAssessed(address indexed buyer, uint256 indexed eventId, uint256 riskScore
    event WalletBlacklisted(address indexed wallet, string reason);
    event ThresholdUpdated(uint256 oldThreshold, uint256 newThreshold);

    constructor(address _aiSigner) EIP712("RexellAIOracle", "1") {
        aiSigner = _aiSigner;
    }

    function verifyApproval(
        address buyer,
        uint256 eventId,
        uint256 riskScore,
        uint256 deadline,
        bytes calldata signature
    ) external returns (bool) {
        require(!blacklisted[buyer], "Wallet is blacklisted");
        require(block.timestamp <= deadline, "Approval expired");

        bytes32 structHash = keccak256(abi.encode(
            APPROVAL_TYPEHASH,
            buyer,
            eventId,
            riskScore,
            nonces[buyer],
            deadline
        ));

        bytes32 digest = _hashTypedDataV4(structHash);
```

```
        address signer = ECDSA.recover(digest, signature);
        require(signer == aiSigner, "Invalid AI signature");

        nonces[buyer]++;
        lastRiskScore[buyer] = riskScore;

        bool approved = riskScore <= riskThreshold;
        emit RiskAssessed(buyer, eventId, riskScore, approved);

        return approved;
    }

    function blacklistWallet(address wallet, string calldata reason) external onlyOwner {
        blacklisted[wallet] = true;
        emit WalletBlacklisted(wallet, reason);
    }

    function updateThreshold(uint256 newThreshold) external onlyOwner {
        emit ThresholdUpdated(riskThreshold, newThreshold);
        riskThreshold = newThreshold;
    }

    function updateSigner(address newSigner) external onlyOwner {
        aiSigner = newSigner;
    }
}
```

6.2 Modified Rexell.sol buyTicket()

```
// In Rexell.sol - modified buyTicket function
import "./AIOracle.sol";

contract Rexell is ERC721, ERC721URIStorage {
    AIOracle public aiOracle;

    modifier aiApproved(
        uint256 eventId,
        uint256 riskScore,
        uint256 deadline,
        bytes calldata aiSignature
    ) {
        require(
            aiOracle.verifyApproval(msg.sender, eventId, riskScore, deadline, aiSignatu
            "AI risk assessment failed"
        );
        _;
    }

    function buyTicket(
        uint256 eventId,
        uint256 riskScore,
        uint256 deadline,
        bytes calldata aiSignature
    ) external payable aiApproved(eventId, riskScore, deadline, aiSignature) {
        // ... existing buyTicket logic unchanged ...
```

```
        Event storage evt = events[eventId];
        require(evt.ticketsSold < evt.maxTickets, "Sold out");
        require(msg.value >= evt.ticketPrice, "Insufficient payment");

        uint256 tokenId = _nextTokenId++;
        _safeMint(msg.sender, tokenId);
        _setTokenURI(tokenId, evt.metadataURI);
        evt.ticketsSold++;

        emit TicketPurchased(eventId, tokenId, msg.sender);
    }

    // Fallback for low-risk events (organizer can disable AI check)
    function buyTicketNoAI(uint256 eventId) external payable {
        require(!events[eventId].aiEnforced, "AI check required for this event");
        // ... standard buy logic ...
    }
}
```

6.3 SoulboundIdentity Score Updates

After each transaction, the AI system can update the user's SoulboundIdentity KYC score to reflect their behavioral trust history:

```
// Called by AI backend via server wallet after purchase analysis
function updateTrustScore(address user, uint256 newScore) external onlyAIOracle {
    require(hasValidIdentity(user), "No soulbound identity");
    require(newScore <= 100, "Score out of range");
    identities[user].score = newScore;
    emit ScoreUpdated(user, newScore);
}
```

6.4 Gas Optimization Strategy

On-chain AI enforcement adds gas overhead. The design minimizes cost through:

- **Off-chain computation:** All ML inference happens server-side; only a 65-byte ECDSA signature is verified on-chain (~6,000 gas for `ecrecover`).
- **Nonce-based replay protection:** Single `uint256` increment (~5,000 gas) instead of storing full approval structs.
- **Lazy risk score storage:** `lastRiskScore` mapping update is optional and can be skipped for ALLOW decisions to save ~20,000 gas.
- **Estimated additional gas per `buyTicket`:** ~30,000 gas (~0.001 cUSD at current Celo gas prices), a negligible increase.

7. Training Data Strategy

7.1 Synthetic Data Generation

Building on the PureNFT methodology of 500,000 synthetic entries, the training dataset must be expanded to 2 million entries incorporating the new feature dimensions:

```
# scripts/generate_training_data.py
import numpy as np
import pandas as pd
from sklearn.utils import resample

def generate_synthetic_dataset(n_samples=2_000_000):
    # Class distribution (with SMOTE later): 85% human, 10% bot, 5% scalper
    n_human = int(n_samples * 0.85)
    n_bot = int(n_samples * 0.10)
    n_scalper = int(n_samples * 0.05)

    # Human behavior profiles
    human_data = {
        'mouse_speed_mean': np.random.normal(0.5, 0.15, n_human),  # px/ms
        'mouse_jitter': np.random.normal(0.3, 0.1, n_human),
        'keystroke_interval_mean': np.random.normal(150, 40, n_human),  # ms
        'purchase_velocity_s': np.random.lognormal(2.5, 0.8, n_human),  # seconds
        'wallet_age_days': np.random.exponential(180, n_human),
        'tx_count': np.random.exponential(50, n_human),
        'session_duration_s': np.random.lognormal(5.5, 0.6, n_human),
        'label': 0
    }

    # Bot behavior profiles (faster, more regular, less natural)
    bot_data = {
        'mouse_speed_mean': np.random.uniform(0.0, 0.05, n_bot),  # minimal/no movemen
        'mouse_jitter': np.random.uniform(0.0, 0.02, n_bot),
        'keystroke_interval_mean': np.random.normal(50, 10, n_bot),  # too fast
        'purchase_velocity_s': np.random.uniform(0.1, 2.0, n_bot),
        'wallet_age_days': np.random.exponential(5, n_bot),  # fresh wallets
        'tx_count': np.random.exponential(3, n_bot),
        'session_duration_s': np.random.uniform(1, 10, n_bot),
        'label': 1
    }

    # Scalper profiles (human-like but pattern-specific)
    scalper_data = {
        'mouse_speed_mean': np.random.normal(0.55, 0.1, n_scalper),
        'mouse_jitter': np.random.normal(0.28, 0.08, n_scalper),
        'keystroke_interval_mean': np.random.normal(120, 30, n_scalper),
        'purchase_velocity_s': np.random.lognormal(1.0, 0.3, n_scalper),  # faster
        'wallet_age_days': np.random.exponential(60, n_scalper),
        'tx_count': np.random.exponential(100, n_scalper),  # high activity
        'session_duration_s': np.random.lognormal(3.5, 0.4, n_scalper),
        'label': 2
    }
```

```
    df = pd.concat([pd.DataFrame(d) for d in [human_data, bot_data, scalper_data]])
    return df.sample(frac=1).reset_index(drop=True)
```

7.2 Real Data Collection Pipeline

- **Behavioral data**: Collected via the frontend `BehavioralCollector` component and stored server-side in a PostgreSQL/Supabase database. Each session generates ~50 KB of behavioral telemetry.
- **On-chain data**: A Celo indexer service (running as a cron job) fetches transaction history for all wallets that interact with Rexell contracts. Tools like The Graph or Celo Explorer API provide indexed data.
- **Labeling strategy**: Initial labels from the heuristic filter (high confidence only), with manual review by organizers for edge cases. A labeling dashboard allows flagging suspicious purchases post-event.

7.3 Class Imbalance Handling

The PureNFT study uses SMOTE (Synthetic Minority Oversampling Technique) to handle the ~5% scalper class. The pipeline applies:

- **SMOTE** on minority classes (bot, scalper) to achieve 40/30/30 split during training.
- **Focal loss** ($FL(p_t) = -\alpha_t(1-p_t)^\gamma \log(p_t)$) to focus learning on hard-to-classify examples.
- **Stratified k-fold cross-validation** (k=5) ensuring each fold preserves class ratios.

7.4 Continuous Learning Pipeline

```
[Production Traffic] → [Feature Store] → [Weekly Batch Retrain]
                                          ├── Train new model candidate
                                          ├── A/B evaluate vs production model
                                          ├── Auto-promote if accuracy > current + 1%
                                          └── Alert if data drift detected (KS test p <
```

8. Implementation Roadmap

Phase 1: Server-Side Migration & Feature Expansion (Weeks 1–3)

Objective: Move all AI logic server-side and expand the feature set.

Files to create:

- `app/api/ai/assess-risk/route.ts` — Main risk assessment API endpoint
- `app/api/ai/collect-behavior/route.ts` — Behavioral data ingestion
- `app/api/ai/logs/route.ts` — Improved AI logging with database persistence
- `lib/ai/server/feature-store.ts` — Server-side feature storage (Redis)
- `lib/ai/server/risk-pipeline.ts` — Orchestrator replacing client-side `AIModeService`

- `lib/ai/layers/heuristic-filter.ts` — Layer 1 implementation

Files to modify:

- `lib/ai/ai-mode.ts` — Convert to thin client that calls server API
- `lib/ai/models/bot-detector.ts` — Deprecate, replace with server-side
- `lib/ai/models/scalping-detector.ts` — Deprecate, replace with server-side

API interface:

```
// app/api/ai/assess-risk/route.ts
import { NextRequest, NextResponse } from 'next/server';
import { heuristicFilter } from '@/lib/ai/layers/heuristic-filter';
import { getOnChainFeatures } from '@/lib/ai/features/on-chain';
import { featureStore } from '@/lib/ai/server/feature-store';

export async function POST(req: NextRequest) {
  const { walletAddress, eventId, behavioral, device, sessionId } = await req.json();

  // Store behavioral data server-side
  await featureStore.recordBehavior(sessionId, behavioral);

  // Fetch on-chain features
  const onChainFeatures = await getOnChainFeatures(walletAddress);

  // Build feature vector
  const features = {
    behavioral,
    device,
    temporal: await featureStore.getTemporalFeatures(walletAddress),
    onChain: onChainFeatures,
    event: await featureStore.getEventFeatures(eventId),
    network: { ip: req.headers.get('x-forwarded-for') }
  };

  // Layer 1: Heuristic
  const heuristicResult = heuristicFilter(features);
  if (heuristicResult.isBot && heuristicResult.confidence > 0.95) {
    return NextResponse.json({
      action: 'BLOCK',
      riskScore: heuristicResult.confidence,
      reason: heuristicResult.reason
    });
  }

  // Phase 1: Use enhanced rule-based scoring (CNN comes in Phase 3)
  const riskScore = computeEnhancedRiskScore(features, heuristicResult);
  const action = riskScore > 0.7 ? 'BLOCK' : riskScore > 0.3 ? 'WARNING' : 'ALLOW';

  // Generate signed approval if allowed
  let approval = null;
  if (action === 'ALLOW') {
    approval = await generateEIP712Approval(walletAddress, eventId, riskScore);
  }
```

```
  return NextResponse.json({ action, riskScore, approval });
}
```

Phase 2: Behavioral Biometrics & Device Fingerprinting (Weeks 4–6)

Objective: Deploy the frontend data collection layer and device fingerprinting.

Files to create:

- `components/ai/BehavioralCollector.tsx` — React component wrapping event pages
- `components/ai/DeviceFingerprinter.tsx` — Generates stable device hash
- `lib/ai/features/behavioral.ts` — Feature extraction from raw events
- `lib/ai/features/device.ts` — Fingerprint generation and analysis
- `lib/ai/features/network.ts` — IP reputation and proxy detection
- `hooks/useBehavioralTracking.ts` — React hook for event page integration

Key component:

```
// hooks/useBehavioralTracking.ts
import { useEffect, useRef, useCallback } from 'react';

interface BehavioralData {
  mouseEvents: Array<{ x: number; y: number; t: number; type: string }>;
  keyEvents: Array<{ key: string; holdTime: number; interval: number }>;
  scrollEvents: Array<{ delta: number; t: number }>;
  clickEvents: Array<{ x: number; y: number; t: number; targetDist: number }>;
}

export function useBehavioralTracking(sessionId: string) {
  const data = useRef<BehavioralData>({
    mouseEvents: [], keyEvents: [], scrollEvents: [], clickEvents: []
  });
  const lastKeyTime = useRef(0);

  useEffect(() => {
    const onMouseMove = (e: MouseEvent) => {
      data.current.mouseEvents.push({
        x: e.clientX, y: e.clientY, t: Date.now(), type: 'move'
      });
    };
    const onKeyDown = (e: KeyboardEvent) => {
      const now = Date.now();
      const interval = lastKeyTime.current ? now - lastKeyTime.current : 0;
      lastKeyTime.current = now;
      data.current.keyEvents.push({ key: e.key, holdTime: 0, interval });
    };
    const onScroll = (e: Event) => {
      data.current.scrollEvents.push({ delta: window.scrollY, t: Date.now() });
    };

    // Sample mouse events at 60Hz to limit data volume
    let lastSample = 0;
```

```
    const throttledMouse = (e: MouseEvent) => {
      if (Date.now() - lastSample > 16) { lastSample = Date.now(); onMouseMove(e); }
    };

    document.addEventListener('mousemove', throttledMouse);
    document.addEventListener('keydown', onKeyDown);
    document.addEventListener('scroll', onScroll);

    return () => {
      document.removeEventListener('mousemove', throttledMouse);
      document.removeEventListener('keydown', onKeyDown);
      document.removeEventListener('scroll', onScroll);
    };
  }, []);

  const getBehavioralData = useCallback(() => data.current, []);
  const flush = useCallback(async () => {
    await fetch('/api/ai/collect-behavior', {
      method: 'POST',
      body: JSON.stringify({ sessionId, data: data.current })
    });
  }, [sessionId]);

  return { getBehavioralData, flush };
}
```

Phase 3: CNN Model Training & Deployment (Weeks 7–9)

Objective: Train the PureNFT-inspired CNN and deploy it for inference via ONNX Runtime.

Files to create:

- `scripts/ml/train_cnn.py` — Training script using TensorFlow/PyTorch
- `scripts/ml/export_onnx.py` — Export trained model to ONNX format
- `scripts/ml/generate_training_data.py` — Synthetic dataset generation
- `lib/ai/layers/behavioral-cnn.ts` — ONNX Runtime inference wrapper
- `lib/ai/layers/graph-nn.ts` — GNN for coordination detection
- `models/bot_detector.onnx` — Exported model artifact

Model serving:

```
// lib/ai/layers/behavioral-cnn.ts
import * as ort from 'onnxruntime-node';

let session: ort.InferenceSession | null = null;

async function getSession(): Promise<ort.InferenceSession> {
  if (!session) {
    session = await ort.InferenceSession.create('./models/bot_detector.onnx', {
      executionProviders: ['cpu'],
      graphOptimizationLevel: 'all'
    });
```

```
  }
  return session;
}

export async function runCNNInference(featureMap: Float32Array): Promise<{
  human: number; bot: number; scalper: number;
}> {
  const sess = await getSession();
  const tensor = new ort.Tensor('float32', featureMap, [1, 1, 48, 48]);
  const results = await sess.run({ input: tensor });
  const probs = results.output.data as Float32Array;
  return { human: probs[0], bot: probs[1], scalper: probs[2] };
}
```

Phase 4: Blockchain-AI Smart Contract Integration (Weeks 10–12)

Objective: Deploy `AIOracle.sol`, modify `Rexell.sol`, and wire up the full pipeline.

Files to create:

- `contracts/AIOracle.sol` — Oracle contract (see Section 6.1)
- `scripts/deploy-oracle.ts` — Hardhat deployment script
- `lib/ai/server/eip712-signer.ts` — Server-side EIP-712 approval signing
- `test/AIOracle.test.ts` — Contract unit tests

Files to modify:

- `contracts/Rexell.sol` — Add `aiApproved` modifier, modify `buyTicket()`
- `contracts/SoulboundIdentity.sol` — Add `updateTrustScore()` function
- Frontend buy flow components — Pass AI signature to contract call

EIP-712 signer:

```
// lib/ai/server/eip712-signer.ts
import { ethers } from 'ethers';

const AI_SIGNER_KEY = process.env.AI_SIGNER_PRIVATE_KEY!;

const domain = {
  name: 'RexellAIOracle',
  version: '1',
  chainId: 42220, // Celo mainnet
  verifyingContract: process.env.AI_ORACLE_ADDRESS!
};

const types = {
  AIApproval: [
    { name: 'buyer', type: 'address' },
    { name: 'eventId', type: 'uint256' },
    { name: 'riskScore', type: 'uint256' },
    { name: 'nonce', type: 'uint256' },
    { name: 'deadline', type: 'uint256' }
```

```
  ]
};

export async function signApproval(
  buyer: string, eventId: number, riskScore: number, nonce: number
): Promise<{ signature: string; deadline: number }> {
  const wallet = new ethers.Wallet(AI_SIGNER_KEY);
  const deadline = Math.floor(Date.now() / 1000) + 300; // 5 min validity
  const scaledRisk = Math.floor(riskScore * 1000); // Scale to integer

  const signature = await wallet.signTypedData(domain, types, {
    buyer, eventId, riskScore: scaledRisk, nonce, deadline
  });

  return { signature, deadline };
}
```

Phase 5: Testing, Optimization & Documentation (Weeks 13–14)

Objective: Comprehensive testing, performance tuning, and documentation.

Activities:

- End-to-end integration testing across all layers
- Adversarial testing with simulated bot attacks
- Gas profiling on Celo testnet (Alfajores)
- Latency optimization (target < 200ms total)
- Documentation of API endpoints, model architecture, and deployment procedures
- A/B testing framework setup for gradual rollout

9. New File Structure

```
rexell/
├── app/
│   └── api/
│       └── ai/
│           ├── assess-risk/
│           │   └── route.ts           # Main risk assessment endpoint
│           ├── collect-behavior/
│           │   └── route.ts           # Behavioral data ingestion
│           ├── logs/
│           │   └── route.ts           # AI decision logging
│           ├── dashboard/
│           │   └── route.ts           # Organizer AI dashboard data
│           └── retrain/
│               └── route.ts           # Model retraining trigger (admin)
├── components/
│   └── ai/
│       ├── BehavioralCollector.tsx      # Wraps event pages for data collection
│       ├── DeviceFingerprinter.tsx      # Generates device fingerprint
│       ├── RiskWarningModal.tsx        # Warning UI for moderate-risk users
```

```
│       └── OrganizerAIDashboard.tsx     # Dashboard showing AI decisions
├── contracts/
│   ├── AIOracle.sol                    # NEW: AI approval oracle
│   ├── Rexell.sol                      # MODIFIED: AI-gated buyTicket
│   └── SoulboundIdentity.sol           # MODIFIED: AI trust score updates
├── hooks/
│   ├── useBehavioralTracking.ts        # NEW: Behavioral data collection hook
│   └── useAIRiskAssessment.ts          # NEW: Pre-purchase risk check hook
├── lib/
│   └── ai/
│       ├── layers/
│       │   ├── heuristic-filter.ts     # Layer 1: Rule-based pre-filter
│       │   ├── behavioral-cnn.ts       # Layer 2: ONNX CNN inference
│       │   ├── graph-nn.ts             # Layer 3: GNN coordination detection
│       │   └── ensemble-engine.ts      # Layer 4: Final decision
│       ├── features/
│       │   ├── behavioral.ts           # Behavioral feature extraction
│       │   ├── device.ts               # Device fingerprinting
│       │   ├── network.ts              # IP/proxy analysis
│       │   ├── on-chain.ts             # Celo wallet analysis
│       │   ├── temporal.ts             # Temporal pattern features
│       │   └── event-specific.ts       # Event demand/pricing features
│       ├── server/
│       │   ├── feature-store.ts        # Redis-backed feature storage
│       │   ├── risk-pipeline.ts        # Server-side orchestrator
│       │   ├── eip712-signer.ts        # AI approval signing
│       │   └── model-registry.ts       # Model versioning & A/B testing
│       ├── agents/
│       │   ├── risk-agent.ts           # ENHANCED: Uses ensemble output
│       │   └── policy-agent.ts         # ENHANCED: Supports AI oracle
│       ├── models/                     # DEPRECATED: Replaced by layers/
│       │   ├── bot-detector.ts         # → layers/behavioral-cnn.ts
│       │   └── scalping-detector.ts    # → layers/ensemble-engine.ts
│       ├── ai-mode.ts                  # MODIFIED: Thin client → server API
│       └── logger.ts                   # ENHANCED: DB persistence
├── models/
│   ├── bot_detector.onnx               # Trained CNN model
│   └── graph_model.onnx                # Trained GNN model
├── scripts/
│   └── ml/
│       ├── train_cnn.py               # CNN training pipeline
│       ├── train_gnn.py               # GNN training pipeline
│       ├── export_onnx.py             # Model export
│       ├── generate_training_data.py  # Synthetic data generation
│       └── evaluate.py                # Model evaluation & metrics
└── test/
    ├── ai/
    │   ├── heuristic-filter.test.ts   # Layer 1 unit tests
    │   ├── behavioral-cnn.test.ts     # Layer 2 unit tests
    │   ├── ensemble-engine.test.ts    # Layer 4 unit tests
    │   └── risk-pipeline.test.ts      # Integration tests
    └── contracts/
        └── AIOracle.test.ts           # Smart contract tests
```

10. Key API Endpoints

Endpoint	Method	Description	Auth
`/api/ai/assess-risk`	POST	Pre-purchase risk assessment	Wallet signature
`/api/ai/collect-behavior`	POST	Ingest behavioral telemetry	Session token
`/api/ai/logs`	POST	Log AI decisions	API key
`/api/ai/dashboard`	GET	Organizer AI analytics	Organizer wallet
`/api/ai/retrain`	POST	Trigger model retraining	Admin only

`/api/ai/assess-risk` — Request/Response Schema

Request:

```
{
  "walletAddress": "0x1234...abcd",
  "eventId": 42,
  "sessionId": "sess_abc123",
  "behavioral": {
    "mouseEvents": [{ "x": 100, "y": 200, "t": 1711540000000, "type": "move" }],
    "keyEvents": [{ "key": "a", "holdTime": 85, "interval": 142 }],
    "scrollEvents": [{ "delta": 350, "t": 1711540001000 }],
    "clickEvents": [{ "x": 150, "y": 300, "t": 1711540002000, "targetDist": 3.2 }]
  },
  "device": {
    "canvasHash": "a3f9...",
    "webglRenderer": "ANGLE (NVIDIA GeForce RTX 3060)",
    "audioHash": "b2e1...",
    "screenRes": "1920x1080",
    "colorDepth": 24,
    "timezone": "Asia/Kolkata",
    "languages": ["en-US", "en"],
    "pluginCount": 3,
    "webdriver": false
  }
}
```

Response (ALLOW):

```
{
  "action": "ALLOW",
  "riskScore": 0.15,
  "reason": null,
  "approval": {
    "signature": "0xabc123...def456",
    "deadline": 1711540300,
    "nonce": 7,
    "scaledRiskScore": 150
  },
  "layerDetails": {
```

```
    "heuristic": { "isBot": false, "confidence": 0.05 },
    "cnn": { "human": 0.88, "bot": 0.07, "scalper": 0.05 },
    "gnn": { "coordinationProb": 0.02 }
  }
}
```

Response (BLOCK):

```
{
  "action": "BLOCK",
  "riskScore": 0.94,
  "reason": "HIGH_RISK: Bot-like behavioral pattern with fresh wallet",
  "approval": null,
  "layerDetails": {
    "heuristic": { "isBot": true, "confidence": 0.85 },
    "cnn": { "human": 0.04, "bot": 0.91, "scalper": 0.05 },
    "gnn": { "coordinationProb": 0.72 }
  }
}
```

11. Smart Contract Modifications

The full `AIOracle.sol` contract is provided in Section 6.1. Additional modifications include:

Rexell.sol: Event-Level AI Toggle

Organizers can enable or disable AI enforcement per event, allowing low-risk community events to skip the overhead:

```
struct Event {
    // ... existing fields ...
    bool aiEnforced;  // NEW: whether AI check is required
}

function setAIEnforcement(uint256 eventId, bool enforced) external {
    require(msg.sender == events[eventId].organizer, "Not organizer");
    events[eventId].aiEnforced = enforced;
    emit AIEnforcementUpdated(eventId, enforced);
}
```

Blacklist Integration

```
function buyTicket(uint256 eventId, /* ... */) external payable {
    require(!aiOracle.blacklisted(msg.sender), "Wallet blacklisted");
    if (events[eventId].aiEnforced) {
        require(
            aiOracle.verifyApproval(msg.sender, eventId, riskScore, deadline, sig),
            "AI check failed"
        );
    }
```

```
    // ... proceed with purchase ...
}
```

12. Frontend Integration

12.1 Modified Buy Flow

The existing buy ticket flow is augmented with a pre-purchase risk assessment step:

```
// hooks/useAIRiskAssessment.ts
import { useBehavioralTracking } from './useBehavioralTracking';

export function useAIRiskAssessment() {
  const { getBehavioralData, flush } = useBehavioralTracking(sessionId);

  const assessAndBuy = async (eventId: number, walletAddress: string) => {
    // Step 1: Flush behavioral data to server
    await flush();

    // Step 2: Request risk assessment
    const res = await fetch('/api/ai/assess-risk', {
      method: 'POST',
      body: JSON.stringify({
        walletAddress,
        eventId,
        sessionId,
        behavioral: getBehavioralData(),
        device: await generateDeviceFingerprint()
      })
    });
    const assessment = await res.json();

    // Step 3: Handle decision
    if (assessment.action === 'BLOCK') {
      throw new Error(`Purchase blocked: ${assessment.reason}`);
    }
    if (assessment.action === 'WARNING') {
      const proceed = await showRiskWarningModal(assessment);
      if (!proceed) return null;
    }

    // Step 4: Execute on-chain purchase with AI signature
    const tx = await rexellContract.buyTicket(
      eventId,
      assessment.approval.scaledRiskScore,
      assessment.approval.deadline,
      assessment.approval.signature,
      { value: ticketPrice }
    );

    return tx;
  };
```

```
  return { assessAndBuy };
}
```

12.2 Organizer AI Dashboard

A new dashboard component allows event organizers to monitor AI decisions in real time:

- **Risk distribution chart**: Histogram of risk scores for their events.
- **Blocked attempts log**: Table of blocked purchases with reasons.
- **Model confidence metrics**: Average CNN confidence scores.
- **Suspicious wallet clusters**: Visual graph of GNN-detected coordination patterns.
- **Override controls**: Ability to whitelist wallets or adjust per-event risk thresholds.

12.3 BehavioralCollector Integration

```
// In the event page layout
import { BehavioralCollector } from '@/components/ai/BehavioralCollector';

export default function EventPage({ params }: { params: { id: string } }) {
  return (
    <BehavioralCollector sessionId={sessionId} eventId={params.id}>
      {/* Existing event page content */}
      <EventDetails />
      <TicketPurchaseSection />
    </BehavioralCollector>
  );
}
```

13. Performance & Security Considerations

13.1 Latency Budget

The total risk assessment must complete within 200ms to avoid degrading the purchase experience. The layered approach enables **early termination**: if Layer 1 detects an obvious bot with >95% confidence, Layers 2–3 are skipped entirely, reducing average latency to ~15ms for trivial cases. GeeTest's commercial solution achieves 1.4s verification including interactive challenges; the passive approach targets 10× faster performance.

13.2 Anti-Bypass Measures

Attack Vector	Countermeasure
Modifying client-side code	All decisions are server-side; client only collects data
Replaying behavioral data	Session-bound tokens + nonce-based approval expiry

Attack Vector	Countermeasure
Fabricating mouse events	Statistical distribution analysis detects artificial patterns; GAN-generated movements still lack micro-jitter characteristics
Using multiple wallets	GNN Layer 3 detects funding source correlation
Rotating IPs/devices	On-chain behavioral fingerprint persists across sessions
MITM on AI API	HTTPS + EIP-712 signature cannot be forged without server private key
Replay of AI approval	Nonce increments per wallet; approval expires in 5 minutes
Gas manipulation	`AIOracle` verification is mandatory for AI-enforced events

13.3 Privacy Considerations

- **Behavioral data**: Stored server-side with automatic 30-day TTL. No personally identifiable information (PII) beyond wallet address.
- **Device fingerprints**: Stored as one-way hashes; raw fingerprint data is not persisted.
- **On-chain data**: Public by nature (Celo is a public blockchain). No additional privacy concern.
- **GDPR compliance**: Behavioral data collection requires user consent. A consent banner is shown on first visit, and users can opt out (with the trade-off that AI assessment relies more heavily on on-chain and device features).
- **Data minimization**: Only features relevant to risk assessment are collected. Raw event streams are aggregated into feature vectors within 60 seconds and raw data is discarded.

13.4 Gas Optimization

The on-chain overhead is minimal by design:

- `ecrecover`: ~3,000 gas
- Storage reads (nonce, blacklist, threshold): ~6,000 gas (3 × `SLOAD`)
- Storage write (nonce increment): ~5,000 gas (`SSTORE` from non-zero to non-zero)
- Optional risk score storage: ~20,000 gas (`SSTORE` from zero)
- **Total overhead**: ~14,000–34,000 gas per transaction (~0.0005–0.001 cUSD)

14. Testing Strategy

14.1 Unit Tests

Component	Test Cases	Framework
Heuristic filter	WebDriver detection, speed thresholds, datacenter IP matching, edge cases	Jest
CNN inference	Known bot/human feature maps, latency benchmarks, ONNX compatibility	Jest + ONNX
Ensemble engine	Weight correctness, threshold boundaries, hard override logic	Jest

Component	Test Cases	Framework
AIOracle.sol	Signature verification, nonce management, blacklisting, expiry, access control	Hardhat/Chai
Feature extractors	Behavioral feature normalization, device hash stability, on-chain data parsing	Jest

14.2 Integration Tests

- **End-to-end purchase flow**: Simulate a full purchase from behavioral collection through AI assessment to on-chain execution on Celo Alfajores testnet.
- **Cross-layer communication**: Verify data flows correctly between all four layers.
- **Error handling**: Test graceful degradation when Redis is down, ONNX model fails to load, or Celo RPC is unavailable.

14.3 Adversarial Testing

Following the GAN evasion methodology, adversarial testing should include:

- **Selenium/Puppeteer bots**: Automated purchase attempts with headless browsers.
- **GAN-generated mouse movements**: Synthetic human-like trajectories to test Layer 2 robustness.
- **Sybil attacks**: Coordinated purchases from 50+ wallets funded from a single source to test GNN detection.
- **Replay attacks**: Intercepting and replaying valid AI approval signatures.
- **Timing attacks**: Rapidly submitting purchases to test rate limiting and nonce management.

14.4 A/B Testing Plan

Deploy the new system in shadow mode first:

1. **Week 1 (Shadow)**: Run the new pipeline in parallel with the existing system. Log decisions but do not enforce. Compare decision distributions.
2. **Week 2 (5% traffic)**: Route 5% of purchases through the new AI-gated flow. Monitor false positive rate (legitimate users blocked).
3. **Week 3 (25% traffic)**: Scale up if false positive rate < 1%.
4. **Week 4 (100% traffic)**: Full rollout with the old system as fallback.

Success metrics:

- Detection accuracy > 95% (measured against manually labeled test set)
- False positive rate < 0.5% (legitimate users incorrectly blocked)
- Average assessment latency < 200ms (p99 < 500ms)
- On-chain gas overhead < 40,000 gas per purchase

- Zero successful replay attacks in adversarial testing

14.5 Performance Benchmarks

Metric	Current System	Target
Bot detection accuracy	~60% (estimated, hardcoded weights)	> 95%
Scalper detection accuracy	~40% (single-feature rules)	> 90%
False positive rate	Unknown	< 0.5%
Detection bypass difficulty	Trivial (client-side)	Multi-layer evasion required
Enforcement mechanism	Advisory only	On-chain mandatory
Feature dimensions	2 (time delta, frequency)	48+ features across 7 categories
Latency	~1ms (client-side)	< 200ms (server-side with ML)

15. Comparison with Existing Solutions

Approach	Detection Method	Accuracy	On-Chain Integration	Source
Current Rexell	Hardcoded logistic regression	~60% (est.)	None	Rexell GitHub
PureNFT	Lightweight CNN (FCC+MSCGC)	93.57%	Transaction analysis only	IEEE ICCE 2025
Multi-stage pipeline	SGAN + DGCNN	~96%	None	arXiv 2412.02266
BotDetect	Federated learning on EVM	~94%	Smart contract orchestration	arXiv 2501.12112
BotShape	Behavioral time series	98.52%	None	arXiv 2303.10214
Proposed Rexell v2	4-layer pipeline + on-chain oracle	> 95% (target)	Full (AIOracle + SoulboundID)	This plan

The proposed architecture uniquely combines the PureNFT model's lightweight CNN design for real-time inference, the multi-stage pipeline's layered approach for progressive filtering, BotShape's behavioral biometrics for evasion-resistant detection, and BotDetect's blockchain integration pattern for on-chain enforcement—addressing the research gap of architecturally embedding AI decisions into smart contracts rather than treating them as separate analytical components. Anti-scalping strategies for 2025 emphasize the importance of combining multiple detection methods, and CAPTCHA alternatives from Radware confirm that passive behavioral analysis provides better user experience than interactive challenges.

This integration plan transforms Rexell from a platform with advisory-only, client-side bot detection into one with production-grade, multi-layered AI enforcement embedded directly into its smart contract architecture—a significant contribution for a BE capstone project that bridges the gap between AI research and blockchain application.